MTH2005
Case Study 1

Jonas Brown

Introduction

Throughout this report, I will be answering the questions provided in the brief using theory and numerical methods. I will be referring to the code I have written, which has been condensed into one executable file included in the Appendix. You can also find the link to the original GitHub code there, where it is formatted as a Python module that can be imported and used like I have done in the upcoming examples. Also, although I have done my best to format the code in a somewhat pretty way, please excuse the sometimes iffy display from the `listings` package in LaTeX.

Fixed-point method

We were given the equation $f(x) = (x+5)^{1/3} - x + 3 = 0$ in the interval $x \in [3, 7]$ and must use the fixed-point method to find a root, x_*, for any $g(x)$ that converges and verify the root has been found within a numerical error - plotting our results.

Our $g(x)$ functions (iteration functions) are defined as:

- $g_1(x) = (x+5)^{1/3} + 3$,
- $g_2(x) = (x-3)^3 - 5$.

2.1 Convergence of iteration functions

For some $g_n(x)$ to converge, the following conditions must be met:

- $f(x)$ must be continuous on the closed interval $[3, 7]$,
- $f(x)$ must be differentiable on the open interval $(3, 7)$,
- there exists $\xi \in (3, 7)$ that brackets the root,
- and some $|g_n'(\xi)| < 1$.

Is $f(x)$ continuous on $[3, 7]$?

Proof: $(x+5)$ and $(-x+3)$ are polynomials, so they are continuous everywhere. The function $a(x) = x^{1/3}$ is continuous $\forall\ x \in \mathbb{R}$, hence $a(x+5)$ is continuous $\forall\ x \in \mathbb{R}$. So, $a(x+5) + (-x+3)$ is continuous $\forall\ x \in \mathbb{R}$. Therefore, $f(x)$ is continuous on the closed interval $[3, 7]$.

Is $f(x)$ differentiable on $(3, 7)$?

Proof: $(x+5)$ and $(-x+3)$ are polynomials, so they are differentiable everywhere. The function $a(x) = x^{1/3}$ is differentiable $\forall\ x \in \mathbb{R}$ with $x \neq 0$. $x \in (3, 7)$, so $x + 5 \in (8, 12)$ and therefore $x + 5 \neq 0$. Hence, $a(x+5)$ is differentiable $\forall\ x \in \mathbb{R}$. So, $a(x+5) + (-x+3)$ is differentiable $\forall\ x \in \mathbb{R}$. Therefore, $f(x)$ is differentiable on the open interval $(3, 7)$.

Does $(3, 7)$ bracket a root?

This is simple enough to prove. The signs of the values of $f(x)$ must be opposite for the minimum and maximum of the interval $(3, 7)$.

- $f(3) = (3+5)^{1/3} - 3 + 3 = 8^{1/3} = +2$
- $f(7) = (7+5)^{1/3} - 7 + 3 = 12^{1/3} - 4 \approx -1.71$

The two ends of our interval have opposite signs, hence there is a root in $(3, 7)$.

Do the iterable functions converge?

For a function $g_n(x)$ to converge $|g_n'(x)| < 1$, so:

- Does $g_1(x) = (x_n + 5)^{1/3} + 3$ converge?

  Proof:

  $$g_1'(x) = \frac{1}{3}(x_n + 5)^{-2/3}$$

  $(x + 5)^{-2/3}$ is decreasing for $x > 0$, so its maximum on $[3, 7]$ occurs at $x = 3$.

  $$\max_{x \in (3,7)} g_1(x) = \frac{1}{3}(3 + 5)^{-2/3} = \frac{1}{3} \cdot 8^{-2/3} = \frac{1}{3} \cdot \frac{1}{4} = \frac{1}{12}$$

  So, $\forall\ \xi \in (3, 7)$ we have that $|g_1'(\xi)| < 1$ and $g_1(x)$ converges.

- Does $g_2(x) = (x_n - 3)^3 - 5$ converge?

  Proof:

  $$g_2'(x) = 3(x_n - 3)^2$$

  $(x - 3)^2$ is increasing for $x > 0$, so its maximum on $[3, 7]$ occurs at $x = 7$.

  $$\max_{x \in (3,7)} g_2(x) = 3(7 - 3)^2 = 3(4)^2 = 3 \cdot 16 = 48$$

  So, $\exists\ \xi \in (3, 7)$ where $|g_2'(x)| \geq 1$, hence $g_2(x)$ does not converge.

Conclusion

We have found that a root exists in $[3, 7]$ and that $g_1(x)$ can be used to find it, as it will converge to x_*.

2.2 Fixed-point iteration

Proving that $g_2(x)$ diverges

First, let's quickly look at $g_2(x)$:

$$\begin{aligned}
&x_{n+1} = g_2(x_n) = (x_n - 3)^3 - 5 \\
&x_0 = 6 \\
&x_1 = g_2(6) = (6 - 3)^3 - 5 = 3^3 - 5 = 22 \\
&x_2 = 6854 \\
&x_3 \approx 3.2156 \times 10^{11} \\
&\vdots
\end{aligned}$$

We can see that, as n increases, x_n exponentially increases showing numerically that $g_2(x)$ does not converge to x_*.

Finding the root with $g_1(x)$

Now, let's look at $g_1(x)$:

$$\begin{aligned}
&x_{n+1} = g_1(x_n) = (x_n + 5)^{1/3} + 3 \\
&x_0 = 6 \\
&x_1 = g_1(6) = (6 + 5)^{1/3} + 3 = 11^{1/3} + 3 \approx 5.2240 \\
&x_2 \approx 5.1704 \\
&x_3 \approx 5.1666 \\
&\vdots \\
&x_{10} \approx 5.1663
\end{aligned}$$

We can see that as n increases, $x \to x_*$.

Verifying the root

Before we get too excited, let's check that what we have found is our root:

$$x_* = x_{10} = 5.1663$$
$$f(x_*) = (x_* + 5)^{1/3} - x + 3 \approx -2.46 \times 10^{-12} \approx 0$$
$$|f(x_{10}) - f(x_9)| = -3.4637 \times 10^{-11} - -4.8765 \times 10^{-10} \approx 4.5301 \times 10^{-10} \approx 0$$

Code used to find the root

I ran the following in my terminal (usage covered in Fixed-point method under Usage of modules):

```
C:\User\CaseStudy> python main.py fixed-point ./images
Root: 5.1663
Error: 0.0000
```

Producing these graphs:

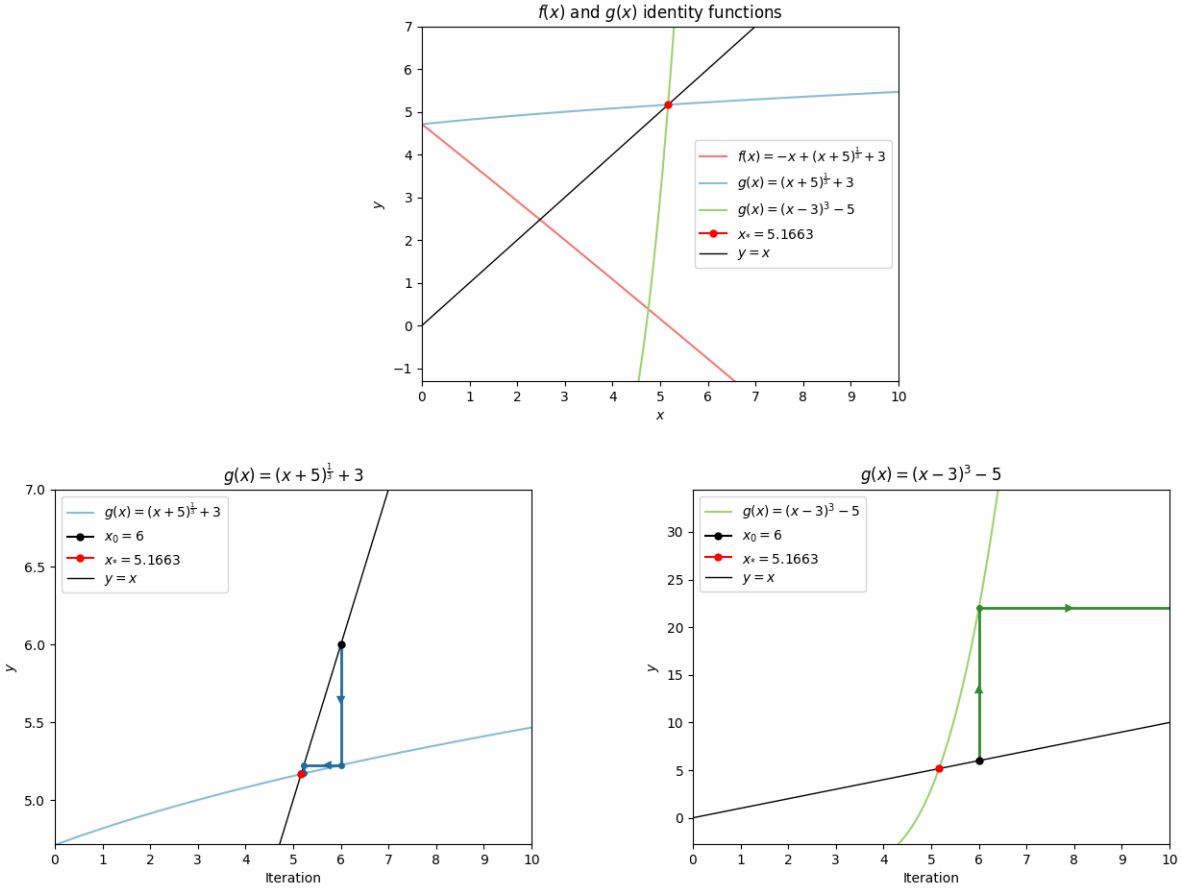


Figure: $f(x)$, $g_1(x)$, $g_2(x)$, $y = x$, and $g_n(x)$ iterations plotted.

2D Newton-Raphson

We are given that $F_{main}(x,y) = 3x^3 + 2y^3 + 2x^2 + 2xy + 2y^2 - 6x - 6y$, and the vector gradient ∇F_{main} and the stationary points are defined as

$$\nabla F_{main} = \begin{pmatrix} 9x^2 + 4x + 2y - 6 \\ 6y^2 + 2x + 4y - 6 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

In order to find the stationary point of F_{main}, we need to solve $\nabla F_{main}(x,y) = 0$.

3.1 Deriving our iterative function

$$\text{Let } \mathbf{x}_n = \begin{pmatrix} x_n \\ y_n \end{pmatrix},\ \nabla F_{main}(\mathbf{x}_n) = \begin{pmatrix} \frac{\partial F}{\partial x} \\ \frac{\partial F}{\partial y} \end{pmatrix} = \begin{pmatrix} f(x_n, y_n) \\ g(x_n, y_n) \end{pmatrix}$$
$$\text{then, } H(x_n, y_n) = \begin{pmatrix} \frac{\partial^2 F}{\partial x^2} & \frac{\partial^2 F}{\partial x \partial y} \\ \frac{\partial^2 F}{\partial y \partial x} & \frac{\partial^2 F}{\partial y^2} \end{pmatrix} = \begin{pmatrix} 18x + 4 & 2 \\ 2 & 12y + 4 \end{pmatrix}$$

We can rearrange the Newton-Raphson identity to equal 0,

$$\begin{aligned} & x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \\ \implies & x_{n+1} \cdot f'(x_n) = x_n \cdot f'(x_n) - f(x_n) \\ \implies & f'(x_n)(x_{n+1} - x_n) = -f(x_n) \\ \implies & f(x_n) + f'(x_n)(x_{n+1} - x_n) = 0 \end{aligned}$$

and then, with $\nabla F_{main}(x_n, y_n) = \mathbf{F}(\mathbf{x}_n)$, we can work back to the Newton-Raphson identity, but with matrices this time.

$$\begin{aligned} & \mathbf{F}(\mathbf{x}_n) + H(\mathbf{x}_n)(\mathbf{x}_{n+1} - \mathbf{x}_n) = 0 \\ \implies & H(\mathbf{x}_n)(\mathbf{x}_{n+1} - \mathbf{x}_n) = -\mathbf{F}(\mathbf{x}_n) \\ \implies & \mathbf{x}_{n+1} - \mathbf{x}_n = -H^{-1}(\mathbf{x}_n)\mathbf{F}(\mathbf{x}_n) \\ \implies & \mathbf{x}_{n+1} = \mathbf{x}_n - H^{-1}(\mathbf{x}_n)\mathbf{F}(\mathbf{x}_n) \end{aligned}$$

Finding $H^{-1}(\mathbf{x}_n)$

We know that for a given matrix

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$
$$A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

So,

$$\begin{aligned} & H(\mathbf{x}_n) = \begin{pmatrix} 18x + 4 & 2 \\ 2 & 12y + 4 \end{pmatrix} \\ & H^{-1}(\mathbf{x}_n) = \frac{1}{(18x + 4)(12y + 4) - 2 \cdot 2} \begin{pmatrix} 12y + 4 & -2 \\ -2 & 18x + 4 \end{pmatrix} \\ \implies & H^{-1}(\mathbf{x}_n) = \frac{1}{216xy + 72x + 48y + 12} \begin{pmatrix} 12y + 4 & -2 \\ -2 & 18x + 4 \end{pmatrix} \end{aligned}$$

3.2 Using our iterative function

We now have that

$$\mathbf{x}_{n+1} = \mathbf{x}_n - H^{-1}(\mathbf{x}_n)\mathbf{F}(\mathbf{x}_n)$$

which is equivilent to

$$\begin{pmatrix} x_{n+1} \\ y_{n+1} \end{pmatrix} = \begin{pmatrix} x_n \\ y_n \end{pmatrix} - \frac{1}{216xy + 72x + 48y + 12} \begin{pmatrix} 12y + 4 & -2 \\ -2 & 18x + 4 \end{pmatrix} \begin{pmatrix} 9x^2 + 4x + 2y - 6 \\ 6y^2 + 2x + 4y - 6 \end{pmatrix}$$

Solving $\nabla F_{main}(x, y) = 0$

The initial condition is

$$\mathbf{x}_0 = \begin{pmatrix} -5 \\ -1 \end{pmatrix}$$

and iterating 10 times yields:

$$\begin{aligned} \mathbf{x}_1 &= \begin{pmatrix} -5 \\ -1 \end{pmatrix} - \frac{1}{684} \begin{pmatrix} -8 & -2 \\ -2 & -86 \end{pmatrix} \begin{pmatrix} 197 \\ -14 \end{pmatrix} = \begin{pmatrix} -2.7368 \\ -2.1842 \end{pmatrix} \\ \mathbf{x}_2 &= \begin{pmatrix} -1.6975 \\ -1.7118 \end{pmatrix} \\ &\vdots \\ \mathbf{x}_{10} &= \begin{pmatrix} -1.2542 \\ -1.5699 \end{pmatrix} \end{aligned}$$

Verifying the root

To verify we have found the root, all we need to do is plug $\mathbf{x}_{10}$ into ∇F_{main}:

$$\nabla F_{main}(\mathbf{x}_{10}) = \begin{pmatrix} 9(-1.2542)^2 + 4(-1.2542) + 2(-1.5699) - 6 \\ 6(-1.5699)^2 + 2(-1.2542) + 4(-1.5699) - 6 \end{pmatrix} = \begin{pmatrix} 5.5876 \times 10^{-4} \\ -4.8394 \times 10^{-4} \end{pmatrix} \approx \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Nature of the stationary point

First, we have to find the value of $H(\mathbf{x}_{10})$,

$$H(\mathbf{x}_{10}) = \begin{pmatrix} 18(-1.2542) + 4 & 2 \\ 2 & 12(-1.5699) + 4 \end{pmatrix} = \begin{pmatrix} -18.5756 & 2 \\ 2 & -14.8388 \end{pmatrix}$$

Then, we calculate the eigenvalues of this matrix as

$$\begin{aligned} \lambda_1 &= -19.4438, \\ \lambda_2 &= -13.9705 \end{aligned}$$

Since they are both negative, the stationary point is a **local maximum**.

Code used to find the root

I ran the following in my terminal (usage covered in 2D Newton-Raphson under Usage of modules):

```
C:\User\CaseStudy> python main.py 2d-nr
Root: [-1.25417304 -1.56992898]
Root verification: [0. 0.]
Hessian eigenvalues: [-19.44380116 -13.9704613 ]
Stationary point nature: local maximum
```

Boundary-value matrix problem

We have been told to consider the boundary-value problem

$$\frac{d^2y}{dx^2} = \cos(x),$$

on the domain $[0, 2\pi]$.

4.1 Finding the true solution

First, let's find a trial solution:

$$\begin{aligned}
&\frac{d^2y}{dx^2} = \cos(x) \\
&\implies \frac{dy}{dx} = \sin(x) + A \\
&\implies y = -\cos(x) + Ax + B
\end{aligned}$$

Now, using the initial conditions $y'(0) = y(2\pi) = 0$,

$$\begin{aligned}
&y'(0) = 0 = \sin(0) + A \\
&\implies A = 0
\end{aligned}$$

$$\begin{aligned}
&y(2\pi) = 0 = -\cos(2\pi) + (0 \cdot 2\pi) + B \\
&\implies B = 1
\end{aligned}$$

So, our true solution is $y_*(x) = -cos(x) + 1$.

4.2 Forming a matrix inversion problem

We start by discretising our domain $[0, 2\pi]$ into N points, giving $x_0, x_1, \ldots, x_{N-1}$ and $\Delta x = \frac{2\pi}{N-1}$. Then, using a finite difference approximation with interior points $i = 1, 2, \ldots, N-2$,

$$\begin{aligned}
&\frac{d^2y}{dx^2} \approx \frac{y_{i-1} - 2y_i + y_{i+1}}{\Delta x^2} \\
&\implies \cos(x_i) = \frac{y_{i-1} - 2y_i + y_{i+1}}{\Delta x^2} \\
&\implies \Delta x^2 \cos(x_i) = y_{i-1} - 2y_i + y_{i+1}
\end{aligned}$$

giving us the linear system

$$T\mathbf{y} = \Delta x^2 \mathbf{f}$$

where T is an $N \times N$ tridiagonal matrix, and $\mathbf{y}$ and $\mathbf{f}$ are vectors:

$$\begin{pmatrix} 1 & 0 & 0 & \ldots & 0 \\ 1 & -2 & 1 & \ldots & 0 \\ 0 & \ddots & \ddots & \ddots & 0 \\ 0 & \ldots & 1 & -2 & 1 \\ 0 & \ldots & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} y_0 \\ \vdots \\ y_i \\ \vdots \\ y_{N-1} \end{pmatrix} = \Delta x^2 \begin{pmatrix} 0 \\ \cos(x_1) \\ \vdots \\ \cos(x_{N-2}) \\ 0 \end{pmatrix}$$

Boundary conditions

We have two boundary conditions to consider on our domain $[0, 2\pi]$:

- Neumann boundary condition on the left boundary:

  The Neumann condition means that we need to prescribe the derivative $y'(0) = 0$ at the boundary. To implement this, we approximate the derivative using a forward Euler approximation:

$$\begin{aligned} y'(0) &\approx \frac{y_1 - y_0}{\Delta x} \\ \implies 0 &= \frac{y_1 - y_0}{\Delta x} \\ \implies 0 &= y_1 - y_0 \\ \implies y_0 - y_1 &= 0 \end{aligned}$$

  Therefore, the first row of our matrix T is $(1, -1, 0, \ldots, 0)$.

- Dirichlet boundary condition on the right boundary:

  The Dirichlet condition means that we need to prescribe the value of the function $y(2\pi) = 0$ at the boundary. Since we're working with a discretised system with the domain split into N points where $x_{N-1} = 2\pi$, the Dirichlet condition applies to the last point.

$$\mathbf{y}_{N-1} = 0$$

  Then, we replace the last row in our matrix T with $(0, 0, \ldots, 0, 1)$ and we ensure $\mathbf{y}_{N-1} = 0$ by setting $\mathbf{f}_{N-1} = 0$.

These conditions give us the linear system

$$\begin{pmatrix} 1 & -1 & 0 & \ldots & 0 \\ 1 & -2 & 1 & \ldots & 0 \\ 0 & \ddots & \ddots & \ddots & 0 \\ 0 & \ldots & 1 & -2 & 1 \\ 0 & \ldots & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} y_0 \\ \vdots \\ y_i \\ \vdots \\ y_{N-1} \end{pmatrix} = \Delta x^2 \begin{pmatrix} 0 \\ \cos(x_1) \\ \vdots \\ \cos(x_{N-2}) \\ 0 \end{pmatrix}$$

4.3 Solving our linear system

To solve our linear system for $\mathbf{y}$ we rearrange $T\mathbf{y} = \Delta x^2 \mathbf{f}$ to equal $\mathbf{y} = T^{-1}\Delta x^2 \mathbf{f}$. To do this I used the `numpy.linalg` Python module to solve our equation. This gave us N plotted points that estimate the solution of the boundary-value problem

$$\frac{d^2y}{dx^2} = \cos(x).$$

Which when placed on a graph alongside the analytical true solution $y_* = -\cos(x) + 1$ looks like

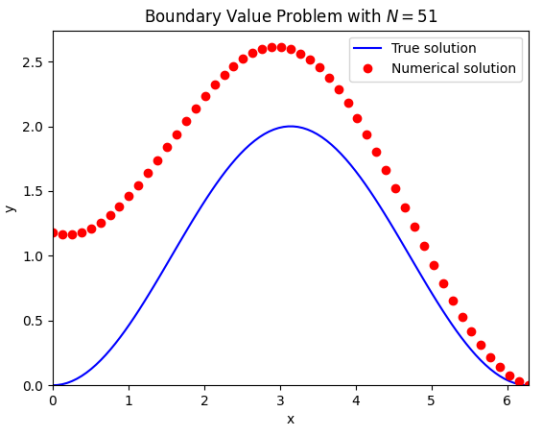


Figure: Numerical solution with $N = 51$.

4.4 The relationship between error and Δx

The error of a numerical solution with grid length Δx is given by

$$\epsilon = \overline{|y_{num} - y_*|},$$

where y_{num} is the numerical solution and y_* is the true solution $y_* = -\cos(x) + 1$. Plotting the error ϵ against Δx on a log-log scale gives:

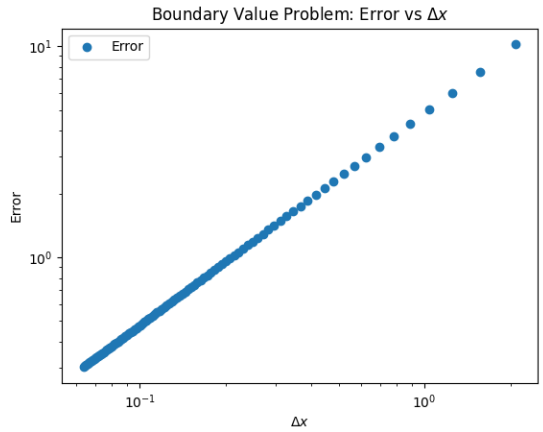


Figure: Error values plotted against Δx on a log-log scale.

As you can see, the line is roughly $y = x$, and so we can say that the error in the numerical solutions satisfies the relation

$$\epsilon \sim \Delta x^n,$$

where n is the order of convergence.

From the log-log plot, the slope is $n \approx 1$, inferring first-order convergence. This is what we expected to see, because:

- The Neumann boundary condition was approximated by a forward Euler scheme - a first-order scheme,
- The interior points use a second-order central difference for y'', but the error ϵ is dominated by the lowest-order term.

Thus, the method exhibits **first-order accuracy**.

Code used to solve the system

I ran the following in my terminal (usage covered in Boundary-value matrix problem under Usage of modules):

```
C:\User\CaseStudy> python main.py boundary
Order: 1.00
```

Appendix

5.1 Code

The code for the report has been condensed into one executable file. The original GitHub project is linked here.

Usage of modules

The code has been structured in a way that allows ease of use and reusability.

Below is a file called `main.py`, it is an example use of the `case_study` module and a shorthand for running any code for the questions:

```python
import sys

from case_study import *

"""
This script demonstrates the usage of various numerical methods
from the case_study module. It can perform fixed-point iteration,
2D Newton-Raphson method, solve boundary value problems,
and condense modules into a single file based on user input.
"""

# Check if this script is being run directly.
if __name__ == "__main__":
    # Get the argument from command line or prompt the user for input.
    arg = sys.argv[1] if len(sys.argv) > 1 else input(
        """Please type "fixed-point", "2d-nr", "boundary", or "condense": """)
    match arg:  # match-case statement to handle different cases based on the input argument.
        case "fixed-point":  # Fixed Point Method case.
            x_0: number = 6  # Initial guess.
            iterations: int = 10  # Number of iterations.

            f: MathFunc = LambdaMathFunc(lambda x: ((x + 5) ** (1 / 3)) - x + 3)  # Function
                definition.
            g_1: MathFunc = LambdaMathFunc(lambda x: ((x + 5) ** (1 / 3)) + 3)  # First identity
                function.
            g_2: MathFunc = LambdaMathFunc(lambda x: ((x - 3) ** 3) - 5)  # Second identity function
                .

            x = np.linspace(0, iterations, 100)  # x values for plotting.
            # Initialise Fixed Point Method.
            method = FixedPointMethod(
                func=f,  # Function to find root of.
                identities=[g_1, g_2],  # Identity functions.
            )

            method.plot(  # Plot the results.
                x=x,  # x values.
                x_0=x_0,  # Initial guess.
                out=sys.argv[2] if len(sys.argv) > 2 else None,  # Output file if provided.
            )

            print(f"Root: {method.root:0.4f}")  # Print the root found.
            print(f"Error: {method.error:0.4f}")  # Print the error.
        case "2d-nr":  # 2D Newton-Raphson Method case.
            f: MathFunc = LambdaMathFunc(lambda x, y: (9 * x ** 2) + (4 * x) + (2 * y) - 6)  #
                Function definition.
            g: MathFunc = LambdaMathFunc(
                lambda x, y: (6 * y ** 2) + (2 * x) + (4 * y) - 6)  # Second function definition.
            # Initial guess as a NumPy array.
            x_0: NPArray = np.array([
                [-5],
                [-1],
            ])
            # Initialise 2D Newton-Raphson Method.
            newton = NewtonRaphson2D(
                f=f,  # First function.
                g=g,  # Second function.
                symbols='x y',  # Variable symbols.
            )

            newton.iterate(  # Perform iterations.
                x_0=x_0,  # Initial guess.
                iterations=10  # Number of iterations.
            )

            newton.analysis()  # Analyse the results.

            print(f"Root: {newton.root}")  # Print the root found.
            print(f"Root verification: {newton.verification}")  # Print verification of the root.
            print(f"Hessian eigenvalues: {newton.eigenvalues}")  # Print Hessian eigenvalues.
            print(f"Stationary point nature: {newton.nature}")  # Print nature of stationary point.
        case "boundary":  # Boundary Value Problem case.
            domain = [0, 2 * np.pi]  # Define the domain.
            # Initialise Boundary Value Problem.
            boundary = BoundaryValueProblem(
                d2ydx2=LambdaMathFunc(np.cos),  # Second derivative function.
```

```
                domain=domain,  # Domain of the problem.
                initialConditions=[  # Initial conditions.
                    [2 * np.pi, 0],  # y(2  ) = 0.
                    [0, 0],  # y'(0) = 0.
                ],
            )
            boundary.plot(  # Plot the solution for N = 51.
                N=51,  # Number of points.
                out=sys.argv[2] if len(sys.argv) > 2 else None  # Output file if provided.
            )
            boundary.loglog(  # Log-log plot for convergence analysis.
                minN=4,  # Minimum number of points.
                maxN=100,  # Maximum number of points.
                out=sys.argv[2] if len(sys.argv) > 2 else None  # Output file if provided.
            )

            print(f"Order: {boundary.order:0.2f}")  # Print the order of convergence.
        case "condense":  # Condense module case.
            output_file = sys.argv[3] if len(sys.argv) > 3 else "."  # Check for output file
                argument.
            # Ensure the output file has a .py extension.
            output_file = os.path.join(output_file, "output.py") if not output_file.endswith(".py")
                else output_file
            condense_module_to_file(  # Condense module to file.
                sys.argv[2] if len(sys.argv) > 2 else None,  # Module name from command line
                    argument.
                output_file,  # Output file path.
            )
        case _:  # Default case for unknown input.
            raise ValueError("Unknown input")  # Raise an error for unknown input.
```

Fixed-point method

Running this in your console will execute the fixed-point method code and then save the plots to the output directory provided.

```
python main.py fixed-point <output-directory>
```

For example:

```
C:\User\CaseStudy> python main.py fixed-point ./images
Root: 1.2345
Error: 0.1234
```

2D Newton-Raphson

Running this in your console will execute the 2D Newton-Raphson code.

```
python main.py 2d-nr
```

For example:

```
C:\User\CaseStudy> python main.py 2d-nr
Root: [-1.2345, 1.2345]
Root verification: [0. 0.]
Hessian eigenvalues: [-12, 34]
Stationary point nature: saddle point
```

Boundary-value matrix problem

Running this in your console will execute the Boundary-value matrix problem code and then save the plots to the output directory provided.

```
python main.py boundary <output-directory>
```

For example:

```
C:\User\CaseStudy> python main.py boundary ./images
Order: 1.00
```

Condensing the module

Running this in your console will take the input directory, condense the module into a single file, and save it to the output directory provided.

```
python main.py condense <input-directory> <output-directory>
```

For example:

```
C:\User\CaseStudy> python main.py condense ./case_study ./output.py
```

Condensed module code

I would put the following code into `case_study.py` for the `main.py` code from above to work directly with this. Or you could remove the line that imports `case_study.py` and paste the contents of main.py directly under the condensed code.

```
import os # Import necessary standard and third-party libraries.
import colorsys
import sympy as sp
import numpy as np
from fractions import Fraction
import matplotlib.pyplot as plt
from typing import Callable, Any, Protocol, runtime_checkable # Define necessary type aliases and
    protocols.

type number = int | float # Define custom type aliases.
type CMapFunc = Callable[[int], ColorValue]
type ColorValue = tuple[float, float, float, float]
type FunctionReturn = FunctionArgument | sp.core.add.Add
type FunctionArgument = NPArray | number | np.float64 | sp.Expr
type NPArray = np.ndarray[tuple[Any, ...], np.dtype[np.float64]]


class BoundaryValueProblem:
    """
        Class to solve a boundary value problem of the form d y/dx  = f(x) with given boundary
            conditions.
    """
    # Mapping from NumPy functions to SymPy functions.
    numpy_to_sympy = {
        np.sin: sp.sin,
        np.cos: sp.cos,
        np.tan: sp.tan,
        np.exp: sp.exp,
        np.log: sp.log,
        np.sqrt: sp.sqrt,
        np.abs: sp.Abs
    }

    @staticmethod
    def convert_numpy_to_sympy(np_func: Callable[..., Any], var='x') -> sp.Expr:
        """
            Convert a NumPy function to a SymPy expression.
            Args:
                np_func (Callable[..., Any]): The NumPy function to convert.
                var (str): The variable name to use in the SymPy expression.
            Returns:
                A SymPy expression corresponding to the NumPy function.
        """
        x = sp.symbols(var) # Define the variable.
        # Get the corresponding SymPy function if defined in our map.
        sp_func = BoundaryValueProblem.numpy_to_sympy.get(np_func)
        if sp_func is None: # If not found:
            return np_func # Return the original NumPy function.
        return sp_func(x) # Return the SymPy expression.

    def __init__(self, d2ydx2: MathFunc, domain: list[number], initialConditions: list[list[number
        ]]):
        """
            Initialise the BoundaryValueProblem instance.
            Taking any second derivative function and boundary conditions to find a solution.
            Again, specific to the Boundary-value matrix problem in the case study, but can
            be used to solve other problems of similar nature.
            Args:
```

```
            d2ydx2 (MathFunc): The second derivative function d y/dx = f(x).
            domain (list[number]): The domain of the problem.
            initialConditions (list[list[number]]): The boundary conditions in order [[x1, y], [
                x2, y'], [x3, y'']].
        """
        self._d2ydx2 = Function(d2ydx2, symbols='x') # Wrap second derivative function.
        self._domain = domain # Store the domain.
        self._initialConditions = initialConditions # Store the boundary conditions.
        self.trueSolution = None # Placeholder for the true solution.
        self.findTrueSolution() # Find the true solution symbolically.
        self.numericalSolution = None # Placeholder for the numerical solution.
        self.error = 0 # Placeholder for the error.
        self.order = 0 # Placeholder for the order of accuracy.

    def findTrueSolution(self):
        """
            Find the true solution symbolically using SymPy.
            This method integrates the second derivative function twice and applies the boundary
                conditions
            to solve for the constants of integration.
        """
        x = sp.symbols('x') # Define the variable.
        C1, C2 = sp.symbols('C1 C2') # Define constants of integration.
        sym_d2ydx2 = self.convert_numpy_to_sympy(self._d2ydx2.func) # Convert the second derivative
            function to SymPy.
        sym_dydx = sp.integrate(sym_d2ydx2, x) + C1 # First integration + constant C1.
        sym_y = sp.integrate(sym_dydx, x) + C2 # Second integration + constant C2.
        functions = [ # List of functions to apply boundary conditions to.
            sym_y,
            sym_dydx,
            sym_d2ydx2,
        ]
        equations = [ # Create equations based on boundary conditions.
            sp.Eq(
                functions[index].subs(x, condition[0]), # Substitute x with boundary condition x-
                    value.
                condition[1] # Set equal to boundary condition y-value.
            ) for index, condition in enumerate(self._initialConditions) # Iterate over each of the
                boundary conditions.
        ]
        solutions = sp.solve(equations, (C1, C2)) # Solve for constants C1 and C2.
        sym_y = sym_y.subs(solutions) # Substitute constants back into the y solution to obtain the
            true solution.
        self.trueSolution = Function(sp.lambdify(x, sym_y, "numpy"), symbols='x') # Wrap the true
            solution as a Function.

    def solveNumerically(self, N: int = 51) -> NPArray:
        """
            Solve the boundary value problem numerically using finite difference method.
            Args:
                N (int): The number of grid points.
            Returns:
                NPArray: The numerical solution array.
        """
        deltaX = self._domain[1] / (N - 1) # Calculate grid spacing  x .
        x = np.linspace(*self._domain, N) # Create x values in domain with grid spacing.
        f = (deltaX**2) * self._d2ydx2(x) # Evaluate f(x) scaled by  x  .
        T = np.zeros((N, N)) # Initialise the tridiagonal matrix T.
        # Fill in the tridiagonal matrix T with finite difference coefficients (1, -2, 1)
        for index in range(1, (N - 1)):
            T[index, (index - 1)] = 1 # Lower diagonal y_{i - 1}.
            T[index, index] = -2 # Main diagonal y_{i}.
            T[index, (index + 1)] = 1 # Upper diagonal y_{i + 1}.
        # Apply Neumann boundary conditions to T: y'(0) = 0 implies y_0 - y_1 = 0.
        T[0, 0] = 1 # => y_0 coefficient (1).
        T[0, 1] = -1 # => y_1 coefficient (-1).
        # Apply Dirichlet boundary conditions to T: y(2  ) = 0 implies y_{N - 1} = 0.
        T[-1, -1] = 1 # Set last row to enforce y_{N - 1} = 0.
        f[-1] = 0 # Set the last entry of f to 0 as y_{N - 1} = 0.
        self.numericalSolution = np.linalg.solve(T, f) # Solve the linear system T * y = f for y.
        # Calculate the mean absolute error between the numerical solution and true solution if
            available.
        self.error = np.mean(np.abs(self.numericalSolution - self.trueSolution(x))) if self.
            trueSolution is not None else 0
        return self.numericalSolution # Return the numerical solution.

    def plot(self, N: int = 51, out: str | None = None, margin: float = 1.1):
```

```
        """
            Plot the numerical and true solutions of the boundary value problem.
            Args:
                N (int): The number of grid points for the numerical solution.
                out (str | None): The output directory to save the plot. If None, the plot is not
                    saved.
                margin (float): Margin factor for y-axis limits.
        """
        self.solveNumerically(N) # Solve numerically to get the solution with a given N.
        figure, axis = plt.subplots(1) # Create a new figure and axis for plotting.
        trueX = np.linspace(*self._domain, 100) # Generate x values for true solution plot.
        self.trueSolution.plot( # Plot the true solution.
            x=trueX, # The x values for true solution.
            axis=axis, # Axis to plot on.
            label="True solution", # Label for the true solution.
            color="blue", # Colour of the true solution.
        )
        # Generate x values for numerical solution plot (same as used in numerical solution).
        numericalX = np.linspace(*self._domain, len(self.numericalSolution))
        axis.plot( # Plot the numerical solution as red circles.
            numericalX, self.numericalSolution, # x and y values from numerical solution.
            "ro", # Red circles for numerical solution points.
            label="Numerical solution", # Label for the numerical solution.
        )
        axis.set_xlabel("x") # Label x-axis.
        axis.set_ylabel("y") # Label y-axis.
        true_solutions = self.trueSolution(trueX) # Evaluate true solution for y-limits calculation.
        # Determine y-limits for better visualisation with a margin adjustment.
        y_min = min(np.nanmin(self.numericalSolution), np.nanmin(true_solutions)) * margin
        y_max = max(np.nanmax(self.numericalSolution), np.nanmax(true_solutions)) * margin
        axis.set_xlim(*self._domain) # Set x-limits to the domain.
        axis.set_ylim(y_min, y_max) # Set y-limits based on calculated min and max.
        axis.set_title(f"Boundary Value Problem with $N = {N}$") # Set plot title.
        axis.legend() # Show legend.
        figure.show() # Display the plot.
        # Check if output directory is provided and exists, then save the figure.
        if type(out) is str and os.path.exists(out):
            figure.savefig(os.path.join(out, f"boundary-value-problem-N-{N}"))

    def loglog(self, minN: int = 4, maxN: int = 100, out: str | None = None):
        """
            Plot the error of numerical solutions with different N values against their
             x  on a log-log scale to determine the order of accuracy.
            Args:
                minN (int): The minimum number of grid points to consider.
                maxN (int): The maximum number of grid points to consider.
                out (str | None): The output directory to save the plot. If None, the plot is not
                    saved.
        """
        errors = [] # List to store errors for different N values.
        for N in range(minN, maxN + 1): # Iterate over N values from minN to maxN.
            self.solveNumerically(N) # Solve numerically for the current N.
            errors.append(self.error) # Append the calculated error to the list.
        errors = np.array(errors) # Convert errors list to a NumPy array for processing.
        figure, axis = plt.subplots(1) # Create a new figure and axis for plotting.
        N = np.linspace(minN, maxN, (maxN - minN)) # Generate N values for x-axis.
        deltaX = self._domain[1] / (N - 1) # Calculate corresponding  x  values.
        # Perform linear fit on log-log data to find the order of accuracy - from the slop of the
            polyfit.
        self.order, _ = np.polyfit(np.log(deltaX), np.log(errors), 1)
        axis.scatter( # Plot the error vs  x  on a log-log scale.
            deltaX, errors, # x and y values.
            label="Error", # Label for the error points.
        )
        axis.set_xlabel("$\\Delta x$") # Label x-axis.
        axis.set_ylabel("Error") # Label y-axis.
        axis.set_xscale("log") # Set x-axis to logarithmic scale.
        axis.set_yscale("log") # Set y-axis to logarithmic scale.
        axis.set_title("Boundary Value Problem: Error vs $\\Delta x$") # Set plot title.
        axis.legend() # Show legend.
        figure.show() # Display the plot.
        # Check if output directory is provided and exists, then save the figure.
        if type(out) is str and os.path.exists(out):
            figure.savefig(os.path.join(out, "boundary-value-problem-loglog"))


class FixedPointMethod:
```

```
    """
        Class to represent the Fixed Point Method for finding roots of equations.
    """

    def __init__(self, func: MathFunc, identities: list[MathFunc]):
        """
            Initialise the Fixed Point Method with a function and its identity functions.
            Args:
                func (MathFunc): The function for which to find the fixed point.
                identities (list[MathFunc]): A list of identity functions g(x) such that x = g(x).
        """
        self.func = Function(func) # Wrap the main function.
        self.root = None # To store the computed root.
        self.error = 0 # To store the error estimate.
        self._iterables = [IterationFunction(identity) for identity in identities] # Wrap identity
            functions.

    def plot(self, x: NPArray, x_0: number, iterations: int = 10, percentile: int = 20, out: str |
        None = None):
        """
            Plot the Fixed Point Method iterations and functions.
            Args:
                x (NPArray): The range of x values to plot.
                x_0 (number): The initial guess for the fixed point.
                iterations (int): The number of iterations to perform.
                percentile (int): The percentile for y-axis limits.
                out (str | None): The output directory to save plots, if any.
        """
        lines = len(self._iterables) + 1 # Number of lines to plot (function + iterations).
        segments = (lines * 2) - 1 # Number of color segments needed.
        figures = [plt.subplots(1) for _ in range(lines)] # Create subplots.
        all_functions_axis = figures[0][1] # Axis to display all functions.
        # Create a colormap for plotting.
        cmap = CMap(
            cmap="Paired", # Use the "Paired" colormap from matplotlib for matching function and
                iteration points.
            segments=segments, # Number of segments for distinct colors.
        ).darken_desaturate(
            factor=0.9
        ) # Darken and desaturate the colormap for better visibility.
        # Plot the main function f(x).
        self.func.plot(
            x=x, # X values to plot.
            axis=all_functions_axis, # Plot on the combined functions axis.
            # Use the last color for the main function since it will always
            # be an odd colour (so we won't have weird overlap).
            color=cmap(segments - 1),
            label=f"$f(x) = {self.func}$", # Label for the function plot.
        )
        all_functions_axis.set_title("$f(x)$ and $g(x)$ identity functions") # Title for the
            combined plot.
        all_functions_axis.set_xlabel("$x$") # X-axis label.
        all_functions_axis.set_ylabel("$y$") # Y-axis label.
        # Iterate over identity functions.
        for index, identity in enumerate(self._iterables):
            identity.iterate( # Perform iterations for each identity function.
                x_0=x_0, # Initial guess.
                iterations=iterations, # Number of iterations.
                axis=figures[index + 1][1], # Axis for the specific identity function plot.
                color=cmap((2 * index) + 1), # Colour of iteration points.
                markersize=4, # Size of iteration markers.
                lw=2, # Line width for iteration lines.
                zorder=10, # Z-order for layering.
            ).plot( # Plot the identity function on the general plot.
                x=x, # X values to plot.
                axis=all_functions_axis, # Plot on the combined functions axis.
                color=cmap(index * 2), # Colour for the identity function.
                label=f"$g(x) = {identity.func}$", # Label for the identity function plot.
            ).plot( # Plot on its own axis.
                x=x, # X values to plot.
                axis=figures[index + 1][1], # Axis for the specific identity function plot.
                color=cmap(index * 2), # Colour for the identity function.
                label=f"$g(x) = {identity.func}$", # Label for the identity function plot.
            ).title( # Set the title for the identity function plot.
                axis=figures[index + 1][1], # Axis for the specific identity function plot.
            )
        # Average the last x values from all the converging identity functions to estimate the root.
```

```
        self.root = float(np.nanmean([
            identity.x_values[-1]
            for identity in self._iterables
        ]))
        # Calculate the error as the absolute difference between the last two estimates.
        self.error = abs(self.func(self.root) - self.func(float(np.nanmean([
            identity.x_values[-2]
            for identity in self._iterables
        ]))))
        # Iterate over each figure.
        for index, (figure, axis) in enumerate(figures):
            if index != 0: # Skip the combined functions plot.
                axis.plot( # Plot the initial guess point.
                    x_0, x_0, # Initial guess point.
                    color="black", # Colour for the initial guess point.
                    marker='o', # Marker style.
                    markersize=5, # Marker size.
                    zorder=11, # Z-order for layering.
                    label=f"$x_{0} = {x_0}$", # Label for the initial guess point.
                )
                axis.set_xlabel("Iteration") # X-axis label.
                axis.set_ylabel("$y$") # Y-axis label.
            # Plot the computed root on all plots.
            axis.plot(
                self.root, self.root, # The root position.
                color="red", # Colour for the root point.
                marker='o', # Marker style.
                markersize=5, # Marker size.
                zorder=11, # Z-order for layering.
                label=f"$x_{{*}} = {self.root:0.4f}$" # Label for the root point.
            )
            # Plot the line y = x.
            axis.plot(
                x, x, # y = x line.
                color="black", # Colour for the line.
                lw=1, # Line width.
                label="$y = x$", # Label for the line.
            )
            # Get all y-data from the lines in the plot.
            all_y = np.concatenate([line.get_ydata() for line in axis.get_lines()])
            all_y = all_y[np.isfinite(all_y)] # Filter out non-finite values.
            # Determine y-axis limits based on percentiles.
            y_min, y_max = np.percentile(np.vstack(all_y), [percentile, 100 - percentile])
            axis.set_xlim(x.min(), x.max()) # Set x-axis limits.
            axis.set_ylim(y_min.min(), max(y_max.max(), x_0 + 1)) # Set y-axis limits.
            axis.locator_params(axis='x', nbins=iterations) # Set number of x-axis bins.
            handles, labels = axis.get_legend_handles_labels() # Get legend handles and labels.
            unique = dict(zip(labels, handles)) # Remove duplicate labels.
            axis.legend(unique.values(), unique.keys()) # Set the legend.
            figure.show() # Display the figure.
            # Check if output directory is specified.
            if type(out) is str and os.path.exists(out):
                figure.savefig( # Save the figure to the specified output directory.
                    os.path.join(out, "fixed-point-" + ("all-functions" if index == 0 else f"g{index
                        }"))
                )

@runtime_checkable # Allows runtime checking of this protocol.
class MathFunc(Protocol):
    """
        A protocol for mathematical functions.
    """
    def __call__(self, *args: FunctionArgument) -> FunctionReturn:
        """Call the mathematical function with number arguments and return a number."""
        ...

class NewtonRaphson2D:
    """
        Class to perform the Newton-Raphson method in 2D.
    """
    def __init__(self, f: MathFunc, g: MathFunc, symbols: str = 'x y'):
        """
            Initialise the NewtonRaphson2D instance with functions and symbols.
            Specifically set up for the question provided in the case study, but
            other function definitions for the stationary points can be used.
            Args:
                f (MathFunc): The first function.
```

```
            g (MathFunc): The second function.
            symbols (str): The symbols used in the functions.
        """
        self._f = Function(f, symbols=symbols) # Wrap the first function.
        self._g = Function(g, symbols=symbols) # Wrap the second function.
        # Define the vector function F, a combination of f and g.
        self._F = lambda x, y: np.array(
            [
                [self._f(x, y)],
                [self._g(x, y)],
            ]
        )
        self._symbols = sp.symbols(symbols) # Define symbolic variables.
        sym_f: sp.core.add.Add = self._f(*self._symbols) # Symbolic representation of f.
        sym_g: sp.core.add.Add = self._g(*self._symbols) # Symbolic representation of g.
        self._dfdx = sp.lambdify(self._symbols, sym_f.diff(self._symbols[0])) # Partial derivative
            of f with respect to x.
        self._dfdy = sp.lambdify(self._symbols, sym_f.diff(self._symbols[1])) # Partial derivative
            of f with respect to y.
        self._dgdx = sp.lambdify(self._symbols, sym_g.diff(self._symbols[0])) # Partial derivative
            of g with respect to x.
        self._dgdy = sp.lambdify(self._symbols, sym_g.diff(self._symbols[1])) # Partial derivative
            of g with respect to y.
        # Define the Jacobian matrix as a function.
        self._jacobian = lambda x, y: np.array(
            [
                [self._dfdx(x, y), self._dfdy(x, y)],
                [self._dgdx(x, y), self._dgdy(x, y)],
            ]
        )
        self.x_values = [] # List to store the x values during iterations.
        self.root = None # To store the found root after iterations.
        self.verification = None # To store the verification of the root.
        self.eigenvalues = None # To store the eigenvalues of the Hessian at the root.
        self.nature = None # To store the nature of the stationary point.

    def _step(self, x: NPArray) -> NPArray:
        """
            Perform a single step of the 2D Newton-Raphson method.
            Args:
                x (NPArray): The current point in 2D space.
            Returns:
                NPArray: The updated point after one step.
        """
        x1, x2 = x[0][0], x[1][0] # Extract x and y components.
        return x - (np.linalg.inv(self._jacobian(x1, x2)) @ self._F(x1, x2)) # x_new = x - J_inv * F

    def iterate(self, x_0: NPArray, iterations: int = 10):
        """
            Perform multiple iterations of the Newton-Raphson method.
            Args:
                x_0 (NPArray): The initial guess for the root.
                iterations (int): The number of iterations to perform.
        """
        self.x_values = [x_0] # Initialise the list with the initial guess.
        # Perform the specified number of iterations using the previous result to calculate the next
            .
        [self.x_values.append(self._step(self.x_values[-1])) for _ in range(iterations)]
        self.root = self.x_values[-1][::, 0] # Store the final result as the root.

    def analysis(self):
        """
            Analysis of the root found by the Newton-Raphson method, including:
                - Verification of the root,
                - Eigenvalues of the Hessian,
                - and nature of the stationary point.
            Raises:
                ValueError: If iterations have not been run yet.
        """
        if self.root is []: # Check if iterations have been performed.
            raise ValueError("Need to run iterations first.") # Raise error if not.
        # Verify the root by evaluating F at the root.
        self.verification = self._F(self.root[0], self.root[1])[::,0]
        # Calculate the Hessian matrix at the root and its eigenvalues.
        hessian = self._jacobian(self.root[0], self.root[1])
        self.eigenvalues = np.linalg.eigvals(hessian) # Compute eigenvalues.
        # Determine the nature of the stationary point based on eigenvalues.
```

```
        self.nature = "local " + ("maximum" if self.eigenvalues[0] < 0 else "minimum") if np.sign(
            self.eigenvalues[0]) == np.sign(self.eigenvalues[1]) else "saddle point"


class CMap:
    """
        A colormap wrapper that allows for color manipulation.
    """
    def __init__(self, cmap: CMapFunc | str = "prism", segments: int = 1):
        """
            Initialise a CMap with a colormap and number of segments.
            Args:
                cmap (CMapFunc | str): The colormap to use. Can be a string name or a callable.
                segments (int): The number of segments in the colormap.
        """
        # Convert string colormap names to actual colormap objects.
        if type(cmap) is str:
            cmap = plt.get_cmap(cmap) # Get colormap by name from matplotlib.
        self._cmap = cmap # Store the colormap callable.
        self._segments = segments # Store the number of segments.

    def darken_desaturate(self, factor=0.85) -> CMapFunc:
        """
            Darken and desaturate the colormap by a given factor.
            Args:
                factor (float): The factor by which to darken and desaturate the colors.
            Returns:
                CMapFunc: The modified colormap function.
        """
        # Generate a list of colors from the original colormap.
        colors: list[ColorValue] = [self._cmap(index) for index in range(self._segments)]
        # Modify each color.
        for index, color in enumerate(colors):
            h, l, s = colorsys.rgb_to_hls(*color[:-1]) # Convert RGB to HLS.
            l *= factor # Darken.
            s *= factor # Desaturate.
            # Convert back to RGB and update the color.
            colors[index] = (*colorsys.hls_to_rgb(h, l, s), color[-1])
        self._cmap = lambda index: colors[index] # Wrap the new colormap into a function.
        return self._cmap # Return the modified colormap function.

    def __call__(self, index: int) -> ColorValue:
        """
            Make the CMap instance callable, forwarding calls to the underlying colormap.
            Args:
                index (int): The index of the color to retrieve.

            Returns:
                ColorValue: The color at the specified index.
        """
        return self._cmap(index) # Call the underlying colormap function.

class Function:
    """
    A wrapper for mathematical functions to facilitate plotting and string representation.
    """
    LIMIT = 1e16 # Threshold to avoid overflow in numerical evaluations.

    def __init__(self, func: MathFunc, symbols: str = 'x'):
        """
            Creates a Function instance wrapping a given mathematical function.
            Args:
                func (MathFunc): The mathematical function to wrap.
                symbols (str): The variable symbols used in the function for string representation.
        """
        match type(func).__name__:
            case "callable" | "ufunc" | "function":     # Check if func is a built-in function or
                user-defined function,
                self.func = func                        # then directly use the callable.
            case _:                                     # Else assume it's already a Function
                instance
                self.func = func.func                   # and use the underlying function.
        self._symbols = symbols # Store symbols for string representation.

    def _efunc(self, *args: FunctionArgument) -> FunctionReturn:
        """
            Safely evaluates the wrapped function with overflow handling.
```

```
            Handles large values by returning NaN for inputs that would cause overflow
            and returns symbolic expressions as is.
            Args:
                *args (FunctionArgument): Input values for the function.
            Returns:
                FunctionReturn: The evaluated result.
        """
        try:
            value = self.func(*args) # Evaluate the function with given args.
             # Check if the result is a symbolic expression or is an sympy function (like cos, sin,
                 etc).
            if type(value) is sp.core.add.Add or str(type(value)) in sp.functions.__all__:
                return value # Return symbolic expressions as is.
            # Handle numerical results with overflow check.
            return np.where((value <= -self.LIMIT) | (value >= self.LIMIT), np.nan, value)
        except ValueError as e: # Catch ValueErrors during evaluation.
            return np.nan # Return NaN for any evaluation errors.

    def plot(self, x: NPArray, axis: plt.Axes, **kwargs: Any) -> Function:
        """
            Plots the function on a given matplotlib axis using given x values.
            Args:
                x (NPArray): The x values for plotting.
                axis (plt.Axes): The matplotlib axis to plot on.
                **kwargs (Any): Additional keyword arguments passed to matplotlib.
            Returns:
                Function: The current Function instance for method chaining.
        """
        y_values = self._efunc(x) # Evaluate the function at given x values.
        axis.set_xlim(np.nanmin(x), np.nanmax(x)) # Set x-axis limits based on mininum and maximum x
             values.
        y_min, y_max = axis.get_ylim() # Get current y-axis limits.
        # Update y-axis limits based on evaluated y values, ensuring NaNs are ignored and that when
            new lines
        # are added, the y-limits expand to accommodate them.
        y_min, y_max = min(y_min, np.nanmin(y_values)), max(y_max, np.nanmax(y_values))
        axis.set_ylim(y_min, y_max) # Set updated y-axis limits.
        axis.plot(x, y_values, **kwargs) # Plot the function on the axis with additional kwargs.
        return self # Return self for method chaining.

    def title(self, axis: plt.Axes) -> Function:
        """
            Sets the title of the plot axis to the LaTeX representation of the function.
            Only implemented for method chaining.
            Args:
                axis (plt.Axes): The matplotlib axis to set the title on.
            Returns:
                Function: The current Function instance for method chaining.
        """
        axis.set_title(f"$g(x) = {self.func}$") # Set the title with LaTeX formatted function.
        return self # Return self for method chaining.

    @staticmethod
    def convert_exponents_to_rational(expr: sp.Expr) -> sp.Expr:
        """
            Recursively converts all floating-point exponents in a sympy expression to rational
                numbers,
            so they can be represented in LaTeX as fractions instead of decimal points.
            Args:
                expr (sp.Expr): The sympy expression to convert.
            Returns:
                sp.Expr: The modified sympy expression with rational exponents.
        """
        if expr.is_Pow: # Check if the expression is a power.
            base, exp = expr.args # Get base and exponent.
             # If exponent is a float, convert it to a rational number.
            if exp.is_Float:
                frac = Fraction(float(exp)).limit_denominator(1000) # Limit denominator to avoid
                    overly complex fractions.
                # Return the base raised to the rational exponent.
                return base ** sp.Rational(frac.numerator, frac.denominator)
            else:
                # Else recursively convert exponents in the base and exponent.
                return expr.func(base, Function.convert_exponents_to_rational(exp))
        elif expr.args: # If the expression has arguments (like in sums, products, etc).
             # Recursively convert exponents in all arguments.
            return expr.func(
```

```
                *(Function.convert_exponents_to_rational(arg) for arg in expr.args)
            )
        else: # Else return as is.
            return expr

    @staticmethod
    def convert_to_str(func: MathFunc, symbols: str = 'x') -> str:
        """
            Converts a mathematical function to its LaTeX string representation,
            ensuring all exponents are represented as rational numbers.
            Args:
                func (MathFunc): The mathematical function to convert.
                symbols (str): The variable symbols used in the function.

            Returns:
                str: The LaTeX string representation of the function.
        """
        # Create a sympy expression by applying the function to sympy symbols.
        # Then convert any floating-point exponents to rational numbers.
        expr_rational = Function.convert_exponents_to_rational(
            func(
                sp.symbols(symbols)
            )
        )
        # Finally, convert the expression to LaTeX format without root notation.
        return sp.latex(expr_rational, root_notation=False)

    def __str__(self) -> str:
        """
            Returns the LaTeX string representation of the wrapped function.
            Returns:
                str: The LaTeX string representation.
        """
        return Function.convert_to_str(self.func, self._symbols)

    def __call__(self, *args: FunctionArgument) -> FunctionReturn:
        """
            Allows the Function instance to be called like a regular function,
            evaluating the wrapped function with overflow handling.
            Args:
                *args (FunctionArgument): Input values for the function.
            Returns:
                FunctionReturn: The evaluated result.
        """
        return self._efunc(*args) # Evaluate the function with overflow handling.

class IterationFunction(Function):
    """
        Class for performing and visualising iterations of a mathematical function.
        Inherits from the Function class.
    """
    def __init__(self, func: MathFunc):
        """
            Initialises the IterationFunction with a given mathematical function.
            Args:
                func (MathFunc): The mathematical function to be iterated.
        """
        super().__init__(func)
        self.x_values = [] # List to store x values during iterations.

    def iterate(self, x_0: number, iterations: int, axis: plt.Axes, arrow_size: int = 17, **kwargs:
        Any) -> IterationFunction:
        """
            Performs iterations of the function starting from an initial value and visualises the
                process on a given plot axis.
            Args:
                x_0 (number): The initial value to start the iterations.
                iterations (int): The number of iterations to perform.
                axis (plt.Axes): The matplotlib axis on which to plot the iterations.
                arrow_size (int, optional): The size of the arrows used in the plot. Defaults to 17.
                **kwargs (Any): Additional keyword arguments for plotting passed to matplotlib.
            Returns:
                IterationFunction: The instance of the IterationFunction after performing iterations
                     for method chaining.
        """
        self.x_values = [float(x_0)] # Reset x_values with the initial value.
        # Perform iterations and store the results using the previous result to find the next.
```

```
        [self.x_values.append(self._efunc(self.x_values[-1])) for i in range(1, iterations)]
        # Plot the iterations on the provided axis with arrows to show direction.
        for index, x_value in enumerate(self.x_values):
            # Check if the next x value exists and both current and next values are finite.
            if index < len(self.x_values) - 1 and np.isfinite(self.x_values[index + 1]) and np.
                isfinite(x_value):
                x_next = self.x_values[index + 1] # Get the next x value.
                midpoint = (x_next + x_value) / 2 # Calculate the midpoint of the current and next x
                    value.
                half_distance = (x_next - x_value) / 2 # Half the distance between current and next
                    x value.
                delta: number = 0.001 * np.sign(half_distance) # Small delta in the direction of the
                    next point to offset arrows for visibility.
                axis.plot([x_value, x_value], [x_value, x_next], **kwargs) # Vertical line from (
                    x_value, x_value) to (x_value, x_next).
                axis.plot([x_value, x_next], [x_next, x_next], marker='o', **kwargs) # Horizontal
                    line from (x_value, x_next) to (x_next, x_next).
                # Only draw arrows if the distance is significant enough.
                if abs(half_distance) > 0.05:
                    # Draw arrow for vertical segment.
                    axis.annotate(
                        '', # No text for the annotation.
                        xy=(x_value, midpoint + delta), # Arrow head position.
                        xytext=(x_value, midpoint), # Arrow tail position.
                        arrowprops=dict( # Arrow properties.
                            arrowstyle="-|>", # Arrow style.
                            color=kwargs["color"], # Colour from kwargs.
                            lw=0, # Line width.
                            shrinkA=0, # No shrinking at the start.
                            shrinkB=0, # No shrinking at the end.
                            mutation_scale=arrow_size, # Size of the arrow head.
                        ),
                    )
                    # Limit the arrow length for horizontal segment if it exceeds the iteration
                        limit
                    # so that the arrow always displays in the middle of the visible part of the
                        line.
                    if abs(midpoint) > iterations:
                        midpoint = (iterations + x_value) / 2 # Adjust midpoint.
                    # Draw arrow for horizontal segment.
                    axis.annotate(
                        '', # No text for the annotation.
                        xy=(midpoint + delta, x_next), # Arrow head position.
                        xytext=(midpoint, x_next), # Arrow tail position.
                        arrowprops=dict( # Arrow properties.
                            arrowstyle="-|>", # Arrow style.
                            color=kwargs["color"], # Colour from kwargs.
                            lw=0, # Line width.
                            shrinkA=0, # No shrinking at the start.
                            shrinkB=0, # No shrinking at the end.
                            mutation_scale=arrow_size, # Size of the arrow head.
                        ),
                    )
        return self # Return self for method chaining.

class LambdaMathFunc(Function):
    """
    A mathematical function wrapper for lambda expressions.
    """
    def __init__(self, func: Callable[..., number]):
        """
            Initialise the LambdaMathFunc with a callable.
            Args:
                func (Callable[..., number]): A function that takes numerical inputs and returns a
                    numerical output.
        """
        super().__init__(func) # Initialise the base Function class with the provided callable.
```